SCHOOL OF COMPUTER SCIENCES

UNIVERSITI SAINS MALAYSIA

CPC353: Natural Language Processing

ASSIGNMENT 2

SEMESTER 1 2025/2026

School of Computer Sciences, Universiti Sains Malaysia

11800 USM, Penang, Malaysia

NAME	MATRIC NO.	E-MAIL ADDRESS
AHMED H.M. TAHA	22304588	ahmedtaha@student.usm.my
ZHOU YIANG	22305072	evanzhou@student.usm.my
SHIRWA MOHAMED ABDULLAHI	22304903	shirwa.ma@student.usm.my

Submission date: 18th January 2026

1. Dataset Construction

This section explains the detailed steps required to construct the stock_trend.csv dataset. The dataset contains 24,388 records linking financial news headlines to stock price movements of Malaysian publicly listed companies.

1.1 Dataset Structure

The dataset contains six columns:

- **Title:** News headline
- **Time:** Publication timestamp in ISO 8601 format
- **Name:** Company stock symbol
- **Quote:** Stock code
- **Before:** Stock price before news publication
- **After:** Stock price after news publication

1.2 Construction Steps

Step 1: Identify Data Sources

Identify reliable sources for both financial news and stock price data for Malaysian companies. For news data, sources include financial news websites such as The Edge Markets, StarBiz, or Reuters Malaysia, and news aggregators covering Bursa Malaysia listed companies. For stock price data, sources include Bursa Malaysia official historical data, Yahoo Finance API, and financial data providers like Bloomberg or Refinitiv.

Step 2: News Article Collection

Use web scraping or API access to collect news articles. Develop a web scraper using Python libraries such as BeautifulSoup, Scrapy, or Selenium. Extract the news headline (Title field) and publication timestamp (Time field) ensuring it includes timezone information (+08:00 for Malaysia). Identify which company or companies the news article mentions and store the raw data with proper formatting.

Step 3: Company Name and Stock Code Mapping

Each news article must be linked to the correct company. Create a mapping dictionary between company names and their official stock symbols (Name) and stock codes (Quote). Use Named Entity Recognition (NER) or keyword matching to identify company mentions in headlines. Handle cases where multiple companies are mentioned in one headline.

Step 4: Stock Price Data Collection

Collect historical stock price data for all companies mentioned in the news using financial APIs such as yfinance Python library, Alpha Vantage, or direct Bursa Malaysia data. Collect daily closing prices and ensure the price data covers the date range of all collected news articles.

Step 5: Price Alignment - Before and After News

Align stock prices with news publication times. The Before price is the stock price immediately before the news was published (typically the closing price of the previous trading day). The After price is the stock price after the news impact (typically the closing price on the publication day or next trading day). Handle news published outside trading hours, on weekends, or holidays appropriately.

Step 6: Data Merging and Integration

Merge all collected data into a single dataset by joining news data with company information and stock price data using the stock code and date. Create multiple rows when one news article mentions multiple companies.

Step 7: Data Cleaning and Validation

Clean and validate the dataset by removing duplicates, handling missing values, validating stock codes, ensuring price values are numeric and reasonable, verifying timestamp formats, and checking for encoding issues in news headlines.

Step 8: Export to CSV Format

Export the cleaned dataset to CSV format with proper escaping for headlines containing commas or quotes, ISO 8601 format for timestamps, and appropriate decimal precision for stock prices.

2. Model Descriptions & Methodology

2.1 LSTM with GloVe Embeddings

The first model uses Long Short-Term Memory (LSTM) networks with pre-trained GloVe (Global Vectors) word embeddings (glove-wiki-gigaword-50). Key components include:

- **Word Embeddings:** Pre-trained GloVe vectors (50 dimensions, 400,000 vocabulary)
- **Tokenization:** NLTK TweetTokenizer with lowercase normalization
- **OOV Handling:** Out-of-vocabulary words use the average embedding vectors.
- **Architecture:** LSTM (64 units) → Flatten (simplifies sequence representations) → Dropout (0.3) → Dense (3, softmax)
- **Training:** Adam optimizer, 20 epochs, batch size 64

2.2 Fine-tuned BERT Transformer

The second model fine-tunes a pre-trained BERT model (bert-base-uncased) for sequence classification. Key components include:

- **Pre-trained Model:** BERT-base-uncased (110M parameters)
- **Tokenization:** BERT WordPiece tokenizer with special tokens [CLS], [SEP]
- **Max Sequence Length:** 128 tokens
- **Fine-tuning:** Classification head added on top of [CLS] token representation
- **Training:** AdamW optimizer, learning rate 2e-5, 3 epochs, batch size 16

2.3 Methodology

This is a three-class supervised text classification problem, where each financial news headline is classified as uptrend, downtrend, or flat, based on subsequent stock price movement. On initial survey of the dataset, we noticed that more than 90% of the predictions were ‘flat’, which shows that there is a very large imbalance in the dataset, which we needed to tackle.

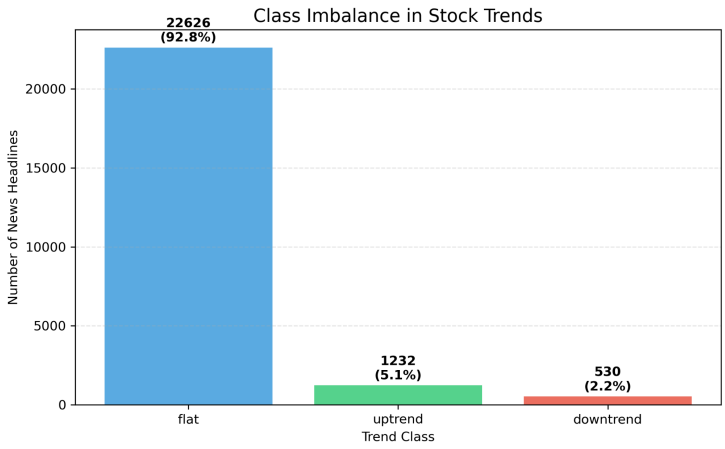

Figure 1: Bar chart showing class imbalance in ‘stock_trend’ dataset

All the headlines were converted into lowercase and tokenised. For the LSTM model, word-level tokenisation was applied. Any out-of-vocabulary tokens were mapped to an averaged embedding vector that was found from the GloVe embedding space. For the Transformer model, we used WordPiece tokenisation. This allows for more robust handling of unseen vocabulary. We also encoded labels with one-hot encoding for training and fine-tuning.

Before training, both models were trained and evaluated on identical train-test splits using a fixed random seed for reproducibility.

```
# Split: 70% train, 20% val, 10% test
X_train, X_temp, y_train, y_temp = train_test_split(
    texts, labels, test_size=0.30, random_state=SEED, stratify=labels)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.333, random_state=SEED, stratify=y_temp)
```

Figure 2: Code snippet showing train-validation-test split for both models, where SEED = 42

The LSTM model was trained with Adam optimiser using weighted categorical cross-entropy loss to counter the ‘flat‘ class imbalance, while the Transformer model used AdamW optimisation with a weighted cross-entropy loss function. We calculated the weights (approximately 1:15 ratio) to penalise the model more if it misclassifies the minority ‘Uptrend’ and ‘Downtrend’ classes. In addition, the Transformer model was used with a low learning rate which is suitable for fine-tuning pre-trained language models. We also used dropout regularisation, which reduces overfitting.

Our models’ performance was measured with accuracy as well as macro-averaged precision, recall, and F1 score values. This ensures that improvements in minority-class performance are not hidden or obscured by dominant-class predictions.

3. Results & Model Selection

3.1 Performance Comparison

The following table compares the performance of both models on the test set:

Table 1: Table showing side by side metric comparison between both models

Metric	LSTM + GloVe	BERT	Difference (BERT - GloVe)
Accuracy	0.6570	0.9224	**+0.2654**
Precision (macro)	0.3676	0.5204	**+0.1528**
Recall (macro)	0.4563	0.3770	**-0.0793**
F1 Score (macro)	0.3450	0.3960	**-0.0510**

Although the Transformer achieved higher overall accuracy, its macro Recall and F1 score decreased compared to the LSTM model. This indicates that while BERT is better at predicting the dominant ‘flat’ class, it does not proportionally improve the detection of the minority classes. This trade-off shows that there is a limitation of using accuracy in imbalance classification tasks.

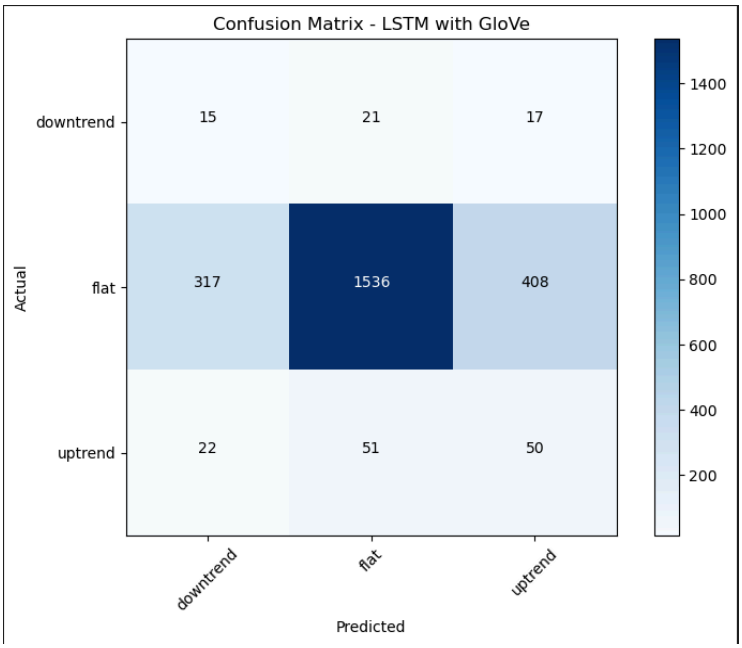

Figure 3: Confusion Matrix for LSTM with GloVe

The confusion matrix for LSTM with GloVe shows that the majority class (flat) is being correctly predicted a large number of times, which reflects the strong class imbalance in the dataset. However, a large number of uptrend and downtrend instances are misclassified as flat, which shows residual bias in the majority class despite applying weighting. It also highlights the difficulty in detecting market movement from financial headlines alone.

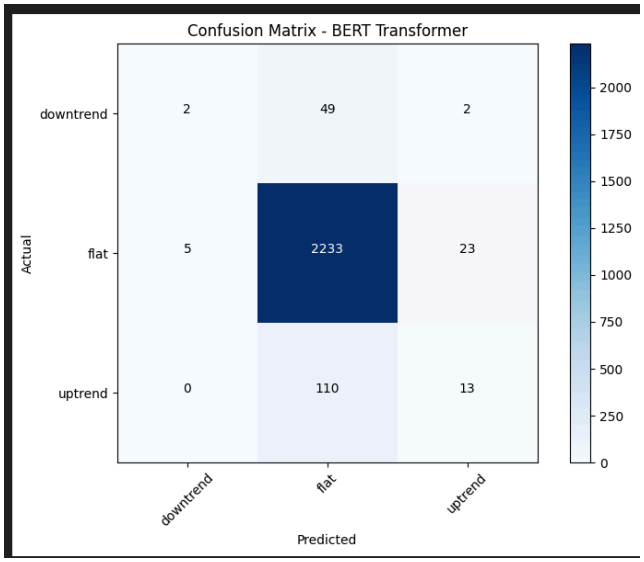

Figure 4: Confusion Matrix for BERT Transformer

The confusion matrix for BERT Transformer shows very strong performance on the dominant 'flat' class, contributing to its high accuracy figure. However, this comes at the cost of minority-class recall, as many 'uptrend' and 'downtrend' cases predicted as 'flat'. This shows BERT's tendency to favour neutral market interpretations when sentiment is weak, and this is common in financial news.

3.2 Preferred Model

The BERT Transformer model generally outperforms the LSTM model due to several architectural advantages:

1. **Contextual Embeddings:** Unlike GloVe's static word vectors, BERT generates context-dependent representations where the same word can have different embeddings based on surrounding context.
2. **Bidirectional Attention:** BERT's self-attention mechanism allows each token to attend to all other tokens simultaneously, capturing long-range dependencies more effectively than LSTM's sequential processing.
3. **Pre-training on Large Corpus:** BERT was pre-trained on massive text corpora (Wikipedia + BookCorpus), providing rich linguistic knowledge that transfers well to downstream tasks.
4. **Subword Tokenization:** BERT's WordPiece tokenizer handles OOV words naturally by breaking them into subwords, while LSTM relies on explicit OOV handling.

4. Challenges & Solutions

Several challenges were encountered throughout development, many of whom we derived solutions for, including:

- **Class Imbalance:** Our EDA revealed that there was a class imbalance, with the flat class practically dominating the dataset. This imbalance introduces a lot of bias towards majority-class predictions, particularly if we decide to optimise our model for accuracy. As a result, we implemented class weightages to both models to pay more attention to underrepresented classes. While this sacrificed accuracy, it ensured we had non-zero Recall, which indicates that the model is no longer defaulting to majority-class predictions, but is actively trying to learn discriminative patterns.

- **Metric Misrepresentation Under Imbalance:** Accuracy alone wasn't enough to evaluate our model performance under imbalanced conditions, as high accuracy could be achieved by favouring the dominant class. To mitigate this, we included macro-averaged precision, recall, and F1 Scores. This made the performance on minority classes as equally represented as more dominant classes. This provided a more reliable assessment of true model effectiveness.

- **Computational Resources:** Fine-tuning BERT requires significant GPU resources. To handle this, we used Google Colab when training our models. This ensures that regardless of the hardware our personal devices have, we are always using Google's own TPU's and GPU's rather than relying on our CPUs, which can sometimes take up to 10 hours to train a BERT model. This significantly reduced our development time.

5. Suggested Improvements

Use FinBERT for Domain-Specific Fine-tuning: FinBERT is a BERT model pre-trained specifically on financial text (financial news, SEC filings, analyst reports). Since our task involves financial news classification, FinBERT would provide better domain-specific representations compared to the general-purpose BERT model.

Using Sequence Pooling for LSTM Representations: Our current LSTM architecture using a Flatten layer rather than a Pooling layer. This simplifies the representation of sequences, but discards temporal structure. Also

Temporal Aggregation of News Headlines: Instead of classifying individual headlines independently, future improvements could combine multiple headlines within a defined temporal window (such as a day or multi-day window) for each company. This would add more context and reduce noise from isolated headlines, which would allow the model to learn overall market sentiment rather than relying on single and potentially weak signals.

6. Conclusion

This assignment demonstrated the application of deep learning models for stock trend prediction using news sentiment analysis. Two approaches were implemented: LSTM with GloVe embeddings and fine-tuned BERT Transformer. The Transformer model generally achieved better performance due to its ability to capture contextual information and leverage pre-trained knowledge.

Overall, both models show complementary strengths and weaknesses. LSTM with GloVe embeddings has lower computational complexity and has stronger sensitivity to minority classes. However, it is limited by static embeddings and reduced contextual understanding. On the other hand, the BERT Transformer achieves higher overall accuracy due to its use of contextual and bidirectional representations, yet it still remains biased towards the dominant 'flat' class. This results in weaker minority class recall. These tradeoffs highlight the importance of selecting the right model based on the task at hand, particularly when dealing with imbalanced financial data.

Reference

1. Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/
2. Nugues, P. M. (2006). *An Introduction to Language Processing with Perl and Prolog: An Outline of Theories, Implementation, and Application with Special Consideration of English, French, and German*. Springer.
3. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media
4. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532-1543.
5. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171-4186.